

EXAM PREPARATION

Sample Questions & Step-by-Step Answers

■ *Focus: ML Ideas & Deep Conceptual Understanding* ■

#	Topic	Q Count
1	Unsupervised Learning & Clustering	5 Qs
2	K-Means Algorithm	5 Qs
3	Dimensionality Reduction & PCA	5 Qs
4	Bayesian & Maximum Likelihood Learning	6 Qs
5	EM Algorithm & GMM	6 Qs
6	Semi-Supervised Learning	5 Qs

Total: 32 Questions | All answers are conceptual (no derivations required)

SECTION 1 — Unsupervised Learning & Clustering

Q1

What is the fundamental difference between supervised and unsupervised learning? Why do we even need unsupervised learning?

Answer

In supervised learning, every training example has a label (Y). The goal is to learn a mapping $f: X \rightarrow Y$ that minimizes expected loss. The model is told the 'right answer' during training.

In unsupervised learning, there are NO labels. The model must discover hidden structure, patterns, or groupings purely from input data X.

■ **Core idea: labeled data is expensive and scarce; unlabeled data is cheap and abundant. Unsupervised learning extracts value from unlabeled data.**

- Use cases: clustering customers, compressing images (PCA), finding latent topics in text, anomaly detection.

■ **Unsupervised learning has no 'correct' answer — evaluation is harder and often domain-dependent.**

Q2

What is clustering? What properties should a good clustering solution have?

Answer

Clustering is the task of grouping data points such that points in the same cluster are more similar to each other than to points in other clusters. It is the core task of unsupervised learning.

■ **Two key properties: (1) Intra-cluster cohesion — points within a cluster should be close/similar. (2) Inter-cluster separation — different clusters should be far/dissimilar.**

- Good clustering: high intra-cluster similarity, low inter-cluster similarity.
- Distance metrics matter: Euclidean distance, cosine similarity, etc. The choice of metric shapes what 'similar' means.

There is no universally correct clustering — the right answer depends on the application and the distance measure chosen.

Q3

Explain the concept of a 'distance metric' in clustering. What properties must a valid distance metric satisfy?

Answer

A distance metric $d(x, y)$ quantifies how dissimilar two points are. In clustering, it determines which cluster a point belongs to.

■ **Four mathematical properties of a valid metric:** (1) **Non-negativity:** $d(x, y) \geq 0$. (2) **Identity:** $d(x, y) = 0$ iff $x = y$. (3) **Symmetry:** $d(x, y) = d(y, x)$. (4) **Triangle inequality:** $d(x, z) \leq d(x, y) + d(y, z)$.

- Euclidean distance: $\sqrt{\text{sum of squared differences}}$ — works well for continuous, low-dimensional data.
- Manhattan distance: $\text{sum of absolute differences}$ — less sensitive to outliers.

■ **Choosing the wrong distance metric can produce meaningless clusters. Always match the metric to the data type and problem.**

Q4

What is the 'curse of dimensionality' and why does it affect clustering?

Answer

The curse of dimensionality refers to the phenomenon where data becomes increasingly sparse as the number of dimensions grows. In high-dimensional spaces, all points become approximately equidistant from each other.

■ **Why it hurts clustering:** distance metrics lose meaning when points are all roughly equally far apart. The concept of 'nearest neighbor' breaks down — you can't tell which cluster a point belongs to.

As dimensions increase, the volume of the space grows exponentially, but the data does not. Most of the space is empty, making clusters hard to detect.

- ✓ Solution 1: Dimensionality reduction (PCA) before clustering. *Dimensionality Reduction (PCA) before clustering.*
- ✓ Solution 2: Use distance metrics appropriate for high-dimensional data (cosine similarity).

■ **This is a key reason why PCA is often applied before K-means in practice.**

Q5

Compare hard clustering vs. soft clustering. Give a real-world example of each.

Answer

Hard clustering assigns each data point to exactly ONE cluster. The assignment is binary — a point either belongs to cluster i or it does not. K-means is a hard clustering algorithm.

Soft (fuzzy) clustering assigns each point a probability or degree of membership to each cluster. A point can partially belong to multiple clusters simultaneously.

■ **Example of hard clustering: Sorting emails into exactly one folder (Spam or Not Spam). Each email gets one definitive label.**

■ **Example of soft clustering: Gaussian Mixture Models (GMM) — a data point near the boundary of two Gaussian components gets non-zero probability for both. EM uses soft assignments (p_{ij} values).**

■ ***Soft clustering is more realistic for real-world data where boundaries between classes are fuzzy. However, it is computationally more expensive.***

SECTION 2 — K-Means Algorithm

Q6

Explain the K-means algorithm step by step. What is it trying to optimize?

Answer

K-means seeks to partition N data points into K clusters such that the total within-cluster variance (sum of squared distances from each point to its cluster centroid) is minimized. This objective is called the inertia or WCSS (Within-Cluster Sum of Squares).

■ **Algorithm Steps:** (1) **Initialize:** randomly choose K points as initial centroids. (2) **Assignment step:** assign each data point to the nearest centroid. (3) **Update step:** recompute each centroid as the mean of all points assigned to it. (4) **Repeat steps 2-3 until centroids do not change (convergence).**

- The assignment step = hard cluster assignment (each point goes to exactly one cluster).
- The update step = mean calculation, hence the name 'K-means'.

■ ***K-means is guaranteed to converge, but NOT to the global optimum. It may converge to a local minimum depending on initialization.***

Q7

What are the main limitations of K-means? How can each be addressed?

Answer

K-means has several important limitations that any serious practitioner must understand:

- Limitation 1 — Must specify K : You must tell the algorithm how many clusters to find. Fix: Use the Elbow Method (plot WCSS vs K and find the 'elbow') or silhouette analysis.
- Limitation 2 — Sensitive to initialization: Different starting centroids @ different final clusters. Fix: K-means++ initialization (spread out initial centroids), or run multiple times and pick best.
- Limitation 3 — Assumes spherical, equal-sized clusters: K-means uses Euclidean distance, so it favors round clusters. Non-spherical data (e.g., rings) will be misclassified. Fix: Use GMM or DBSCAN.
- Limitation 4 — Sensitive to outliers: A single outlier can pull a centroid far from the true cluster center. Fix: Use K-medoids instead.

■ ***The biggest conceptual limitation: K-means is a hard clustering algorithm — it cannot express uncertainty. GMM is the probabilistic (soft) generalization of K-means.***

Q8

How is K-means related to the EM algorithm? What is the deep connection?

Answer

K-means is actually a special case of the EM algorithm applied to a Gaussian Mixture Model with equal, identity covariances and hard (not soft) assignments.

■ **The parallel: K-means Assignment Step = EM E-step (but hard assignment instead of soft probability). K-means Update Step = EM M-step (recomputing the cluster center = the mean).**

In EM/GMM: each data point gets a soft responsibility p_{ij} = probability it came from component i . The new mean is a weighted average of ALL points. In K-means: each point gets a hard assignment (1 or 0). The new centroid is the average of only the assigned points.

■ **K-means = EM with (a) equal weights, (b) equal spherical covariances, (c) hard instead of soft assignments. As variance $\rightarrow 0$, EM approaches K-means behavior.**

- Implication: K-means minimizes WCSS; EM/GMM maximizes log-likelihood. Both alternate between assignment and update.

Q9

You run K-means with $K=3$ on a dataset. After 10 iterations, the centroids stop moving. Does this guarantee you found the best possible clustering? Explain.

Answer

No, convergence does NOT guarantee the globally optimal solution. K-means converges to a LOCAL minimum of the WCSS objective, not necessarily the global minimum.

■ **Why: K-means is not convex. The objective function has multiple local minima. The algorithm is guaranteed to decrease WCSS at each step (it will never get worse), but it will stop at whatever local minimum it first reaches.**

The final result depends entirely on the initial centroid positions. Two different random initializations on the same data can produce very different final clusterings with different WCSS values.

- Best practice: Run K-means multiple times (e.g., 10-20 times) with different random initializations and select the run with the lowest final WCSS.
- K-means++ provides a smarter initialization that tends to find better local minima on average.

■ **For exam: always distinguish between 'converged' and 'optimal'. K-means always converges; it does not always find the optimum.**

Q10

What is the Elbow Method for choosing K? What is its limitation?

Answer

The Elbow Method plots the WCSS (total within-cluster variance) as a function of K. As K increases, WCSS always decreases — adding more clusters always fits the training data better. The 'elbow' is the point where the rate of decrease sharply slows down, suggesting a natural number of clusters.

■ Intuition: When K is too small, adding one more cluster dramatically reduces WCSS. When K is correct, the marginal benefit of adding more clusters drops. At $K=N$ (each point is its own cluster), $WCSS = 0$ but this is useless.

- Limitation 1: The elbow is often ambiguous — the 'bend' is sometimes not sharp and is subjective.
- Limitation 2: It measures reconstruction quality, not cluster meaningfulness. A statistically good K may not be a semantically useful K.

■ *There is no universally correct K. The choice should combine the Elbow Method with domain knowledge.*

SECTION 3 — Dimensionality Reduction & PCA

Q11

What is dimensionality reduction and why is it needed? Distinguish between feature selection and feature extraction.

Answer

Dimensionality reduction is the process of reducing the number of features/dimensions while retaining as much useful information as possible. It addresses the curse of dimensionality, reduces computation, removes noise, and enables visualization.

■ **Feature Selection:** Choose a subset of the original features to keep. No new features are created. Example: dropping irrelevant columns. Simple but may discard correlated information.

■ **Feature Extraction:** Create NEW features (lower-dimensional) that are combinations of the original features. PCA is feature extraction. The new features may not have direct interpretability but capture variance more efficiently.

- Benefit 1: Reduces overfitting — fewer dimensions @ simpler model.
- Benefit 2: Speeds up training significantly.
- Benefit 3: Enables 2D/3D visualization of high-dimensional data.

■ **Trade-off:** Dimensionality reduction always loses some information. The art is in losing the least important information (noise, redundancy) while keeping signal.

Q12

Explain the core idea of PCA (Principal Component Analysis). What is a 'principal component'?

Answer

PCA finds a new coordinate system where the axes (called principal components) are ordered by the amount of variance in the data they explain. The first principal component (PC1) is the direction of maximum variance. PC2 is the direction of maximum remaining variance perpendicular to PC1, and so on.

■ **A Principal Component is a linear combination of the original features. It is a new axis/direction in the original feature space that captures the most spread (variance) in the data.**

Mathematically, principal components are the eigenvectors of the covariance matrix of the data. The corresponding eigenvalues tell you how much variance each component explains.

- Step 1: Standardize the data (subtract mean, divide by std).
- Step 2: Compute the covariance matrix.
- Step 3: Find eigenvectors (directions) and eigenvalues (variances).
- Step 4: Sort by eigenvalue (descending). Keep top K eigenvectors.
- Step 5: Project data onto these K components.

■ **Key constraint: Principal components are orthogonal (perpendicular) to each other — they capture independent sources of variation.**

Q13

What does 'explained variance ratio' mean in PCA? If the first two PCs explain 85% of variance, what does that mean?

Answer

Each principal component captures a certain proportion of the total variance in the data. The explained variance ratio for $PC_i = \text{eigenvalue}_i / \text{sum}(\text{all eigenvalues})$. It tells you how much of the data's 'spread' or 'information content' is captured by that component.

■ **If the first two PCs explain 85% of variance: Projecting the data down to just 2 dimensions retains 85% of the total information (variance). Only 15% of the data's variability is lost.**

- Scree plot: A plot of explained variance ratio vs. component number. It shows how quickly the components explain variance.
- Cumulative explained variance: Sum the ratios in order. Choose K where cumulative variance $\geq$ threshold (e.g., 95%).

Practical implication: If you had 100 features but the first 5 PCs explain 90% of variance, you can work with just 5 numbers instead of 100, losing very little information — a dramatic compression.

■ **PCA is unsupervised — it maximizes variance, not class separability. For supervised tasks, LDA (Linear Discriminant Analysis) may be better.**

Q14

What are the key assumptions of PCA? When does PCA fail?

Answer

PCA makes several important assumptions that must be understood:

■ **Assumption 1 — Linearity:** PCA finds linear combinations of features. If the true structure is non-linear (e.g., a spiral or circle), PCA cannot capture it. Kernel PCA or autoencoders are needed.

■ **Assumption 2 — Variance = Information:** PCA assumes that high-variance directions are the most informative. But sometimes a low-variance direction is critically important for the task (e.g., a rare but important class).

- PCA fails when: data is non-linear, features are categorical (PCA needs continuous data), outliers are present (they distort variance estimates), or when you need interpretability (PC directions are abstract combinations).
- PCA also fails when the goal is class discrimination — it doesn't use labels, so it may compress away class-separating directions.

■ **Important:** PCA is a lossy transformation. The discarded components are gone. You cannot perfectly reconstruct the original data from the reduced representation.

Q15

A student says 'PCA and K-means are completely unrelated.' Do you agree? Explain the relationship.

Answer

This is INCORRECT. PCA and K-means are closely related both mathematically and practically.

■ **Practical connection:** PCA is routinely applied BEFORE K-means to (1) reduce dimensionality and fight the curse of dimensionality, (2) remove noise, (3) make Euclidean distance meaningful again.

Mathematical connection: Both involve finding directions/centers in data. K-means finds cluster centers that minimize within-cluster variance. PCA finds directions that maximize explained variance. Both are essentially variance-minimization/maximization problems on the data.

- Deep result (Ding & He, 2004): K-means clustering is equivalent to a form of PCA in the limit. The cluster indicator matrix from K-means is related to the top eigenvectors of the data matrix.
- Both are sensitive to scale — standardizing features helps both methods.

■ **For exam:** The key practical point is: PCA ® K-means is a common and powerful pipeline. PCA handles the dimensionality, K-means handles the clustering.

SECTION 4 — Bayesian & Maximum Likelihood Learning

Q16

Explain Bayes' theorem in the context of machine learning. What is the role of the prior, likelihood, and posterior?

Answer

Bayes' theorem is the foundation of probabilistic machine learning. It says: $P(h|d) = \alpha * P(d|h) * P(h)$, where d is observed data and h is a hypothesis.

■ **Prior $P(h)$:** Your belief about hypothesis h BEFORE seeing any data. Encodes domain knowledge or assumptions. In the candy example: equal prior (0.2) over all 5 bag hypotheses.

■ **Likelihood $P(d|h)$:** How probable is the observed data IF hypothesis h were true? This is the 'fit' of the hypothesis to the data.

■ **Posterior $P(h|d)$:** Your updated belief about h AFTER observing data d . This is what you actually want — belief updated by evidence.

As you observe more data, the posterior concentrates on the hypothesis that best explains the evidence. In the candy example, after seeing many lime candies, $P(h_5|d) \approx 1$ because h_5 (100% lime bag) explains all observations.

■ **Bayesian learning is exact but expensive — it maintains a distribution over ALL hypotheses. For complex models, this is intractable.**

Q17

What is the difference between MAP (Maximum A Posteriori) and MLE (Maximum Likelihood Estimation)? When does MAP reduce to MLE?

Answer

Both MAP and MLE are ways to find a single 'best' hypothesis parameter rather than maintaining a full posterior distribution.

■ **MLE:** Find θ that maximizes $P(\text{data} | \theta)$ — the likelihood. It ignores the prior. Treats all parameter values as equally likely a priori. Result: $\theta_{\text{MLE}} = \arg\max P(d|\theta)$.

■ **MAP:** Find θ that maximizes $P(\theta | \text{data}) = P(\text{data}|\theta) * P(\theta)$. It incorporates the prior. Result: $\theta_{\text{MAP}} = \arg\max P(d|\theta) * P(\theta)$.

When does MAP = MLE? When the prior $P(\theta)$ is UNIFORM over all hypotheses. A uniform prior contributes equally to all hypotheses, so maximizing $P(d|\theta) * P(\theta) = \text{maximizing } P(d|\theta)$. MAP with uniform prior = MLE.

- MAP is a regularized version of MLE. The prior acts as a regularizer — it pulls estimates toward what was expected before seeing data.

■ **MLE is prone to overfitting with small datasets (e.g., if 3/3 coin flips are heads, MLE says $P(\text{heads})=1.0$). MAP with a suitable prior prevents this.**

Q18

For the candy bag problem, the MLE estimate of theta (fraction of cherry candies) is $\theta = c/N$ (where c = number of cherries observed, N = total). Explain WHY this makes intuitive sense.

Answer

The MLE estimate $\theta = c/N$ is simply the observed relative frequency of cherry candies. Intuitively, the parameter value that makes the observed data 'most likely' is the one that matches what you actually saw.

■ **Derivation intuition:** The log-likelihood is $L = c \cdot \log(\theta) + l \cdot \log(1-\theta)$, where c = cherries, l = limes, $N = c+l$. Taking $dL/d(\theta) = 0$ gives $\theta = c/(c+l) = c/N$.

This is the 'empirical frequency' or 'count-based' estimate. It is the natural thing to do — if you see 7 cherries in 10 draws, your best estimate for $P(\text{cherry})$ is 0.7.

- This extends to the Gaussian: MLE gives μ = sample mean, σ = sample std. Again, the parameter is set to match what the data shows.
- For a Bayesian Network with complete data: MLE for each conditional probability = the observed count fraction in the data.

■ **Limitation:** With small N , MLE is unreliable. If $N=2$ and both are cherry, MLE says $\theta=1.0$ (no limes exist). A Bayesian approach with a prior prevents such extreme estimates.

Q19

Why do we take the LOG of the likelihood in MLE (log-likelihood)? Does this change the answer?

Answer

Taking the log is a mathematical convenience trick that does NOT change the answer because log is a strictly monotonic (increasing) function. The value of θ that maximizes $L(\theta)$ also maximizes $\log L(\theta)$.

■ **Reason 1 — Numerical stability:** The likelihood is a product of many probabilities (each between 0 and 1). Multiplying many small numbers causes numerical underflow (the product rounds to 0 in a computer). Taking log converts products to sums: $\log(\prod p_i) = \sum(\log p_i)$.

■ **Reason 2 — Mathematical simplicity:** Taking derivatives of sums is much easier than taking derivatives of products. The log-likelihood of a Gaussian has a clean closed form.

- For the Gaussian: $\log P(x) = -\log(\sigma \cdot \sqrt{2\pi}) - (x-\mu)^2 / (2 \cdot \sigma^2)$. The log removes the exponential, making calculus easy.

■ **Never forget:** log-likelihood is always NEGATIVE (log of a probability < 1 is negative). Maximizing log-likelihood = minimizing negative log-likelihood (NLL). Many ML frameworks minimize NLL.

Q20

What does it mean that 'maximizing likelihood for a Gaussian with linearly dependent mean is equivalent to minimizing squared error'? Why is this important?

Answer

This is a fundamental result connecting probabilistic models to classical regression. If we model $P(y|x)$ as a Gaussian with mean $\theta_1 x + \theta_2$ (linear) and fixed variance, then maximizing $\log P(y|x)$ w.r.t. θ is equivalent to minimizing $(y - (\theta_1 x + \theta_2))^2$.

■ **Implication: Linear Regression is secretly MLE under the assumption that errors are Gaussian! The 'least squares' loss function is not arbitrary — it arises naturally from the Gaussian likelihood assumption.**

The $\log P(y|x) = \text{constant} - (y - (\theta_1 x + \theta_2))^2 / (2\sigma^2)$. Maximizing this w.r.t. θ is equivalent to minimizing the squared term.

- This unifies two seemingly different approaches: regression (optimization) and probabilistic modeling (likelihood).
- It means choosing a different noise model (e.g., Laplacian instead of Gaussian) leads to a different loss function (L1/absolute instead of L2/squared).

■ **Deep insight: The loss function you choose implicitly assumes a noise distribution. Choosing MSE implicitly assumes Gaussian noise. This is a key ML concept.**

Q21

In the Bayesian candy example, why does the posterior converge to h5 (100% lime bag) as we observe more and more lime candies? What does this tell us about Bayesian learning?

Answer

As more lime candies are observed, the data becomes increasingly unlikely under any hypothesis other than h5 (100% lime). The likelihood $P(d|h5) = 1^N = 1$ for all N lime observations, while for h4 (75% lime), $P(d|h4) = 0.75^N \rightarrow 0$ exponentially fast.

■ **Bayesian learning is self-correcting: the posterior weight of wrong hypotheses shrinks exponentially with data. The correct hypothesis eventually dominates.**

This is why in the graph (from the slides), as N (number of lime observations) increases, $P(h5|d) \rightarrow 1$ while all other hypotheses $\rightarrow 0$. The Bayesian prediction $P(\text{next is lime}|d)$ also $\rightarrow 1.0$.

- With INFINITE data: Bayesian posterior, MAP, and MLE all give the same answer (the true parameter). They differ only with LIMITED data.

■ **The prior matters most when data is SCARCE. With abundant data, the likelihood dominates and prior beliefs wash out. This is why MLE works well in big-data regimes.**

SECTION 5 — EM Algorithm & Gaussian Mixture Models

Q22

What problem does the EM algorithm solve? What is the fundamental 'chicken-and-egg' challenge it addresses?

Answer

The EM algorithm solves the problem of Maximum Likelihood Estimation when there are HIDDEN (latent) variables in the model. In a Gaussian Mixture Model, the hidden variable is which Gaussian component generated each data point.

■ **The chicken-and-egg problem: To find the best GMM parameters (means, covariances, weights), you need to know which component each data point came from. But to know which component each point came from, you need the parameters. You cannot solve for either without the other.**

EM breaks this deadlock by alternating: (E-step) assume current parameters are correct and compute soft assignments. (M-step) assume current soft assignments are correct and re-estimate parameters. Repeat until convergence.

- EM stands for Expectation-Maximization. E-step computes expected assignments; M-step maximizes the expected log-likelihood.

■ **EM is a general framework — not just for GMMs. It applies to ANY model with hidden variables: Hidden Markov Models, mixture models, etc.**

Q23

Explain the E-step and M-step of EM for a Gaussian Mixture Model in plain English.

Answer

Consider a GMM with K Gaussian components. Each component i has weight w_i , mean μ_i , and covariance Σ_i .

■ **E-Step (Expectation):** For each data point x_j and each component i , compute p_{ij} = the probability that data point x_j was generated by component i , given current parameters. This uses Bayes' rule: p_{ij} proportional to $w_i * N(x_j; \mu_i, \Sigma_i)$. The p_{ij} values are 'soft responsibilities' — they sum to 1 across i for each j .

■ **M-Step (Maximization):** Use the p_{ij} values to re-estimate all parameters. $n_i = \sum_j p_{ij}$ (effective number of points for component i). New μ_i = weighted average of all points with weight p_{ij} . New Σ_i = weighted covariance. New $w_i = n_i / N$.

- E-step = soft assignment of data points to components.
- M-step = re-estimate Gaussian parameters using weighted data.

After M-step, go back to E-step with new parameters. This cycle repeats until parameters converge (change by less than a threshold).

■ *The M-step formulas look exactly like standard Gaussian MLE — except each data point is weighted by p_{ij} instead of counted uniformly. This is the core insight.*

Q24

Is the EM algorithm guaranteed to find the global optimum? What is the saddle point problem?

Answer

NO. EM is only guaranteed to find a LOCAL maximum of the log-likelihood, not the global maximum. This is a critical limitation.

■ **Why:** The log-likelihood surface for a mixture model is non-convex — it has multiple local maxima and saddle points. EM follows the gradient and will stop at whatever local maximum it reaches from its starting point.

Log-likelihood is GUARANTEED to be non-decreasing at each EM iteration (it never gets worse). But 'non-decreasing' only means it climbs uphill from the starting point — it does not mean it reaches the highest peak.

- Saddle point problem: EM can also get stuck at saddle points (points where gradient = 0 but it's not a maximum). This is illustrated in the saddle point diagram from the slides.

■ **Solution:** Random restarts. Run EM multiple times from different random initializations. Keep the solution with the highest final log-likelihood.

■ *For GMMs specifically: if two Gaussians become identical (collapse), EM can get stuck. Another issue: a component can collapse to explain just one point, sending its covariance to 0 and log-likelihood to infinity (degenerate solution).*

Q25

What parameters does a GMM have? What is the role of each?

Answer

A Gaussian Mixture Model with K components has the following parameters:

■ **1. Weights w_i (or $P(C=i)$):** The prior probability that a randomly sampled data point came from component i . Must satisfy: $\sum(w_i) = 1$, $w_i \geq 0$. Controls the 'size' or 'importance' of each component.

■ **2. Means μ_i :** The center of each Gaussian component. Controls WHERE the cluster is located in feature space.

■ **3. Covariances Σ_i :** The shape and orientation of each Gaussian component. Diagonal Σ = axis-aligned ellipse. Full Σ = rotated ellipse. $\Sigma = \sigma^2 I$ = sphere.

The joint probability of observing x from component i is: $p(x, y=i | \theta) = w_i \cdot N(x; \mu_i, \Sigma_i)$.

The marginal is: $p(x) = \sum_i w_i \cdot N(x; \mu_i, \Sigma_i)$.

- Total parameters: K weights + K means (each d -dimensional) + K covariances (each $d \times d$) — can be many parameters for high-dimensional data.

■ **Covariance matrix size grows as d^2 . For high-dimensional data, often constrained to diagonal or spherical covariance to avoid overfitting.**

Q26

A colleague claims: 'The log-likelihood always increases in EM, so if I run it long enough, I will eventually get the best possible model.' Evaluate this claim.

Answer

This claim is INCORRECT and demonstrates a common misconception about EM.

■ **True part:** Log-likelihood is guaranteed to be non-decreasing at every EM iteration. Each E-M cycle either increases the log-likelihood or leaves it unchanged. The algorithm will not 'go backward.'

■ **False part:** Running longer does NOT help escape a local maximum. Once EM reaches a local maximum (or saddle point), both the E-step and M-step produce no change, and the algorithm has converged — permanently at a suboptimal point.

The graph from the slides shows log-likelihood increasing and plateauing near the true model's value. But from a different starting point, EM could plateau at a lower value — running it longer would not help.

- The only way to search for a better solution: restart from a different initial point.

■ **Key insight:** EM's convergence guarantee is about monotone improvement from a given starting point, NOT about finding the global optimum. These are completely different guarantees.

Q27

Compare K-means and GMM/EM. In what scenarios would you prefer one over the other?

Answer

K-means and GMM are both clustering algorithms, but they differ in fundamental ways:

- Cluster shape: K-means assumes spherical clusters (equal Euclidean distance). GMM can model elliptical clusters of different sizes and orientations via the covariance matrix.
- Assignment type: K-means = hard (each point belongs to one cluster). GMM = soft (each point has probability distribution over all clusters).
- Probabilistic model: K-means has no probabilistic interpretation. GMM is a full generative probabilistic model — you can compute $P(x)$ and generate new samples.
- Speed: K-means is faster and simpler. GMM (via EM) is slower but more expressive.

■ Prefer K-means when: data has roughly spherical clusters, you need speed, you want simple implementation, data is very high-dimensional.

■ Prefer GMM when: clusters have different shapes or sizes, you need uncertainty estimates in assignments, you want a generative model, you are doing semi-supervised learning (natural EM framework).

■ *GMM subsumes K-means: when all covariances are forced to be equal and spherical ($\Sigma_i = \sigma^2 \cdot I$) and $\sigma \rightarrow 0$, GMM hard assignments approach K-means assignments.*

SECTION 6 — Semi-Supervised Learning

Q28

What is semi-supervised learning (SSL)? What is the fundamental motivation for it?

Answer

Semi-Supervised Learning (SSL) uses BOTH a small set of labeled data $\{(X_i, Y_i)\}$ and a large set of unlabeled data $\{X_j\}$ to learn a better predictor than could be learned from labeled data alone.

■ **Motivation:** In most real tasks, labeled data is expensive and scarce (requires human experts, special equipment, or experiments), while unlabeled data is cheap and abundant. SSL leverages the cheap unlabeled data to improve upon the expensive small labeled set.

- Example: Medical images — millions of MRI scans exist (unlabeled), but radiologist diagnoses (labels) are few and expensive.
- Formally: SSL has $m \gg n$ where n = labeled examples, m = unlabeled examples. The prediction rule $f_{n,m}$ should be better than f_n (trained on labeled only).

SSL works because unlabeled data reveals the structure of the input distribution $P(X)$. If the model form is correct, knowing where data clusters (from unlabeled data) helps determine class boundaries.

■ **SSL is not always beneficial. If the model's distributional assumptions are wrong, unlabeled data can HURT performance. This is a key risk.**

Q29

Explain the Pseudo-Labeling approach to SSL. What are its key steps and what can go wrong?

Answer

Pseudo-labeling is a self-training SSL approach:

■ **Step 1:** Train a model on the small labeled dataset as normal. **Step 2:** Apply this model to the unlabeled data. For each unlabeled point, get a predicted label AND a confidence score (probability). **Step 3:** Select high-confidence predictions (above a threshold, OR take top-K most confident). **Step 4:** Add these pseudo-labeled points to your labeled training set. **Step 5:** Retrain the model on the expanded labeled set. Repeat.

- Threshold approach: Add all unlabeled points where confidence $> p$ (e.g., 0.9).
- Top-K approach: Sort by confidence, add top K predictions regardless of absolute threshold.

■ **What can go wrong:** If the initial model is poor, pseudo-labels will be wrong. Adding wrong labels propagates errors — the model learns from its own mistakes and can drift in the wrong direction (error amplification).

■ **Critical requirement:** The labeled and unlabeled data must come from the SAME distribution. A validation set (fully labeled) should be maintained to check that the model is not going wrong direction.

Q30

Explain the GMM-based (EM) approach to semi-supervised learning. How is it different from purely unsupervised EM?

Answer

The GMM-based SSL approach uses the labeled data to INITIALIZE the EM algorithm instead of random initialization, then runs EM on all data (labeled + unlabeled).

■ **Key difference from unsupervised EM:** In unsupervised EM (on GMMs), we randomly initialize parameters and run E-M loops. In SSL-EM, we first use the labeled data to compute initial parameter estimates (sample mean and covariance per class). Then we run EM on both labeled and unlabeled data together.

The labeled data constrains the model — it tells us which Gaussian belongs to which class. The unlabeled data then refines the shape (covariance) and coverage of each Gaussian. The EM loops update parameters to explain both labeled and unlabeled data jointly.

- Objective being maximized: $p(X_l, Y_l, X_u | \theta)$ — joint likelihood of labeled data with labels AND unlabeled data without labels.
- Performance: SSL-EM outperforms pure supervised EM, requires fewer iterations to converge, and leverages the structure of unlabeled data.

■ **GMM assumption must hold: If data does NOT come from Gaussians, this approach can fail and may even be WORSE than supervised-only learning. This is the key risk of SSL.**

Q31

What is the Co-training algorithm? What are its key assumptions?

Answer

Co-training (Blum & Mitchell, 1998) is an SSL algorithm that works when each data instance has TWO independent views (two subsets of features). Two separate classifiers are trained, one per view, and they teach each other using high-confidence predictions on unlabeled data.

■ **Algorithm:** (1) Initialize $L1 = L2 =$ labeled data. (2) Train classifier $f^{(1)}$ on $L1$ using view-1 features, $f^{(2)}$ on $L2$ using view-2 features. (3) Each classifier labels unlabeled data. (4) $f^{(1)}$ adds its top-k most confident predictions to $L2$. $f^{(2)}$ adds its top-k predictions to $L1$. (5) Remove these from unlabeled pool. (6) Repeat until unlabeled data is exhausted.

■ **Three key assumptions:** (A) Feature split exists: features can be split into two conditionally independent views. (B) Each view alone is sufficient to train a good classifier. (C) The two views are conditionally independent given the class label.

Example (Named Entity Classification): View 1 = the entity words themselves (Mr. Washington vs. Washington State). View 2 = the surrounding context words (vice president of... vs. headquartered in...). Each view provides independent information about the label.

■ **If the two views are not truly independent, co-training can collapse to a single view and fail to learn anything new from unlabeled data.**

A student says 'More unlabeled data always helps in SSL.' Is this true? Under what conditions can unlabeled data actually HURT performance?

Answer

This is FALSE. More unlabeled data does NOT always help. In fact, unlabeled data can actively HURT performance in specific conditions.

■ **When unlabeled data hurts — Model misspecification:** SSL assumes the generative model form $p(X, Y|\theta)$ is correct. If the true data distribution is NOT Gaussian but we use a GMM, then adding unlabeled data makes the model fit a WRONG model better — leading to a worse decision boundary. From the slides: a 'wrong model' can have HIGHER log-likelihood than the correct model on finite data.

The slides show this with a striking example: fitting two separate Gaussians (wrong model) gives higher log-likelihood (-847) than fitting a horizontally elongated Gaussian (correct model, -921). The algorithm confidently learns the wrong thing.

- When unlabeled data hurts — Distribution mismatch: If unlabeled data comes from a different distribution than labeled data, it misleads the model about class boundaries.
- When unlabeled data hurts — Error amplification: In pseudo-labeling, if initial model is poor, wrong pseudo-labels compound errors.

■ **Fix for GMM misspecification:** (1) Use multiple Gaussians per class to better model complex distributions. (2) Down-weight unlabeled data with $\lambda < 1$: add $\lambda * \log p(X_u|\theta)$ instead of full weight. This reduces the damage of wrong assumptions.

■ **Bottom line for exam:** SSL is powerful but risky. It helps when distributional assumptions are correct and hurts when they are wrong. Always validate with held-out labeled data.

QUICK REFERENCE — Key Concepts Cheat Sheet

Concept	One-Line Summary
Supervised Learning	Learn $f: X \rightarrow Y$ from labeled data (X_i, Y_i)
Unsupervised Learning	Find hidden structure in unlabeled data X
Clustering	Group similar points; maximize intra-cluster similarity
K-means	Hard clustering; minimize WCSS; converges to LOCAL minimum
K-means weakness	Spherical assumption; sensitive to init & outliers; K unknown
Curse of Dimensionality	High-dim: all points equidistant; distances lose meaning
PCA	Find directions of max variance; eigenvectors of covariance matrix
Principal Component	Linear combination of features capturing max variance
Explained Variance	Fraction of total variance captured by a PC
Bayesian Learning	$P(h d) = \alpha \cdot P(d h) \cdot P(h)$; posterior over all hypotheses
MLE	Find θ maximizing $P(\text{data} \theta)$; assumes uniform prior
MAP	Find θ maximizing $P(\text{data} \theta) \cdot P(\theta)$; uses prior
MAP vs MLE	MAP = MLE when prior is uniform
Log-Likelihood	Log for numerical stability; monotone so same argmax
Gaussian MLE	μ = sample mean; σ = sample std
MSE = MLE	Minimizing MSE = MLE under Gaussian noise assumption
GMM	Mixture of K Gaussians; parameters: weights, means, covariances
EM E-step	Compute soft assignments $p_{ij} = P(C=i x_j)$ via Bayes
EM M-step	Re-estimate μ_i, σ_i, w_i using weighted data
EM convergence	Log-likelihood increases each iter; converges to LOCAL max
Saddle point	EM can get stuck; fix with random restarts
Semi-Supervised (SSL)	Use small labeled + large unlabeled data; $m \gg n$
Pseudo-labeling	Self-training; add high-confidence unlabeled predictions
SSL-GMM	Initialize with labeled data; run EM on all data
Co-training	Two independent views; classifiers teach each other
SSL risk	Wrong model assumption + unlabeled data @ WORSE performance
SSL fix	Down-weight unlabeled data with $\lambda < 1$

Good luck on your exam! You've got this. ■